통합반 8회차 — 낭독 대본

[이론 01] 기대한 행동을 실제로 검증하는 방법 이해하기

[SLIDE 01] 요구가 바뀌어도 믿고 쓸 수 있는 에이전트의 행동 검증

안녕하세요. 이번 회차에서는 에이전트가 정해 둔 대로 행동하는지, 그걸 어떻게 확인하는지를 다룹니다.

에이전트가 지킬 것은 명세에 적어 두었죠. 그런데 적어 둔 것하고 실제로 일어나는 건 다를 수 있습니다. 요구가 하나 더 붙으면 어떨까요? 그전까지 잘 되던 동작이 그대로인지도 알아야 하고요. 그러면 에이전트가 정말 명세대로 움직이는지, 무엇을 보고 어떻게 확인할 수 있을까요?

답은 기대하는 행동을 시나리오로 적고, 실행으로 되풀이해서 확인하는 데 있습니다. 그 방법을 배우고, 에이전트를 만들고 믿고 맡기기까지 하나씩 더해 온 장치들도 한 번에 정리해 보겠습니다.

[SLIDE 02] 학습목표

이번 회차의 학습목표는 화면의 카드 네 장입니다.

첫 번째 카드는 시나리오예요. 명세에 적은 걸 시나리오로 옮기는데, 전제와 행동과 기대 결과로 나눠 적습니다. 이때 기대 결과는 실행 결과를 보고 맞다, 틀리다를 가릴 수 있는 사실이어야 하고요.

두 번째 카드는 그 시나리오를 실행 가능한 검증으로 잇는 일입니다. 시험을 네 종류로 나눠 보고, 실행할 때마다 출력이 조금씩 달라지는 에이전트에서 무엇을 실패로 셀지 가려 봅니다.

세 번째 카드가 실습이에요. 시나리오를 직접 적고, 되풀이해 실행하고, 실패를 고친 다음 전체를 다시 확인합니다. 여러 출처의 업무 데이터를 엮어서 교대 인수인계서도 만들어 보고요.

마지막 카드는 정리입니다. 에이전트를 만들고 믿기까지의 과정을 쿨러 사건 위에 놓고 한 번에 이어 보고, 응용마다 산출물이 왜 달라지는지도 같이 보겠습니다.

[SLIDE 03] 목차

수업은 화면의 다섯 줄 순서로 갑니다. 위의 세 줄은 행동을 검증하는 방법이에요. 첫 단원에서는 기대하는 행동을 시나리오로 적습니다. 두 번째 단원에서는 그 시나리오를 실행 가능한 시험으로 바꾸고, 중요한 경우를 고르는 네 종류의 시험을 봅니다. 세 번째 단원은 실행할 때마다 출력이 달라지는 에이전트를 어떻게 판정하느냐예요.

넷째 줄이 실습 넷입니다. 앞에서 배운 방법을 직접 해 보는 자리고요. 마지막 단원에서는 에이전트를 만들고 믿기까지 지나온 과정을 쿨러 사건 위에서 하나로 묶겠습니다.

이번 시간에는 위의 세 줄, 기대한 행동을 실제로 검증하는 방법을 배우고, 끝에서 이어질 네 실습의 할 일을 모두 소개합니다.

[SLIDE 04] 기대 행동의 시나리오: BDD와 Given · When · Then

첫 단원은 명세에 적어 둔 약속에서 출발합니다. 승인 전에는 설비 명령을 내지 않는다, 결과는 정해진 항목으로 낸다, 명세에는 이런 약속들이 적혀 있죠. 그런데 이 약속을 어떻게 적어 둬야, 에이전트가 실제로 지켰는지를 실행 결과로 확인할 수 있을까요? 쿨러 사건의 승인 단계에서, 담당자가 추천안을 거절하는 장면 하나를 붙잡고 그 방법을 찾아보겠습니다.

[SLIDE 05] 명세에 적은 것과 실제로 일어나는 것

화면의 그림부터 볼게요. 왼쪽 문서가 명세입니다. 요구사항하고 같이 입력 계약, 출력 계약, 금지 조건, 수용 기준이 한 칸씩 적혀 있죠. 무엇을 받고, 무엇을 내고, 무엇을 하면 안 되고, 언제 다 됐다고 볼지, 이걸 정해 둔 문서예요.

가운데 에이전트가 실제로 실행되면 오른쪽에 결과가 나옵니다. 쿨러 사건이라면 추천안이죠. 그런데 여기서 걸리는 게 하나 있어요. 명세는 문서고, 추천안은 실행해 봐야 나온다는 겁니다. 그러니까 명세를 아무리 잘 적어도, 에이전트가 그대로 움직였는지는 결과를 보기 전엔 알 수 없어요.

명세가 「무엇을 지킬 것인가」에 답했다면, 남은 물음은 「정말 지켰는가」입니다. 그 답이 나오는 곳이 아래쪽 점선이에요. 돋보기가 가리키는 자리, 명세와 실제 결과를 대어 보는 자리죠. 이번 회차에 배울 시나리오와 시험은 모두 이 자리에서 쓰입니다.

[SLIDE 06] 손으로 한 번 확인하는 방식의 한계

화면에서 직접 눌러 확인하면 사용자가 겪는 흐름을 볼 수 있습니다. 다만 한 번 잘 됐다는 기억만으로 변경을 판단하기는 어렵습니다. 승인 단계에도 승인과 거절, 무응답, 권한 없는 요청처럼 여러 경우가 있으니까요.

첫 카드처럼 한 번의 실행은 그중 한 경우를 보여 줍니다. 둘째 카드처럼 기능을 바꾸면 기존 경우도 다시 확인해야 하죠. 그때 무엇을 넣었고 어떤 결과가 맞는지 남기지 않았다면, 셋째 카드처럼 확인하는 사람마다 판단이 달라질 수 있습니다.

손으로 시험해도 입력과 기대 결과를 기록할 수 있습니다. 새로운 문제를 탐색하거나 화면을 이해하기 쉬운지 보는 데도 사람의 확인이 필요해요. 여기서는 반복할 조건을 명확히 남기고 자동으로 다시 확인할 수 있게 만드는 데 초점을 두겠습니다. 사람의 기억에만 기대던 확인을 다음 변경에도 쓸 수 있는 기준으로 바꾸는 겁니다.

[SLIDE 07] BDD: 행동을 시나리오로 적고 실행으로 확인하는 방식

이 문제를 다루는 방식이 BDD, 행동 주도 개발입니다. 업무를 아는 사람과 개발하고 검증하는 사람이 구체적인 사례를 놓고, 어떤 행동이 필요한지 함께 정하는 방식이에요. 먼저 기대를 맞추고, 사례를 시나리오로 적고, 실행 가능한 검증에 연결합니다.

예를 들어 거절한다는 말만 같아도, 한 사람은 승인 화면만 닫힌다고 생각하고 다른 사람은 처리 건까지 종료된다고 생각할 수 있죠. 거절하면 명령은 몇 건이고 처리 상태는 무엇인지 구체적으로

물으면 이 차이가 드러납니다. 코드를 만든 뒤에야 서로 다른 기대를 발견하는 일을 줄이는 거예요.

화면의 행동은 시스템 밖에서 확인할 수 있는 결과를 뜻합니다. 내부에서 어떤 함수를 썼느냐보다, 담당자가 거절했을 때 설비 명령이 생기지 않고 사유가 남는지를 보는 겁니다. 시나리오는 이런 상황과 기대를 구체적인 예로 적은 글이에요.

이번 시간에는 이미 정리한 명세를 바탕으로 시나리오를 만들고 검증에 연결하는 부분을 살펴보겠습니다. 실제 개발에서는 시나리오를 논의하다가 빠진 요구를 찾아 명세를 보완할 수도 있어요.

[SLIDE 08] Given · When · Then: 시나리오의 세 부분

시나리오를 세 부분으로 나누면 무엇을 준비하고 무엇을 확인할지가 보입니다. Given은 행동 전의 전제입니다. 화면에서는 추천안이 올라와 승인 단계가 열려 있는 상태죠. When은 확인할 행동이고, 여기서는 권한 있는 담당자가 사유를 적어 거절하는 일입니다. Then은 그 뒤에 기대하는 결과입니다.

거절 뒤에는 설비 명령과 후속 업무 처리가 없어야 하고, 처리 건은 거절 상태로 닫혀야 합니다. 하나의 거절 행동에도 확인할 결과는 여럿일 수 있어요.

시나리오마다 한 가지 업무 행동에 초점을 맞추면 실패했을 때 원인을 찾기 쉽습니다. 여러 단계가 들어가는 시나리오를 쓸 수 없다는 뜻은 아니에요. 준비 단계와 확인하려는 행동을 섞지 않고, 이 사례에서 무엇을 판단하려는지 드러내는 것이 중요합니다.

[SLIDE 09] 시나리오 한 편: 승인 단계의 거절

앞의 거절을 구조가 보이는 글로 적은 예입니다. 맨 위는 기능 이름, 그 아래는 시나리오 이름이에요. 이어서 전제와 행동, 기대 결과를 적습니다. 이 구조로 시나리오를 쓰는 언어를 Gherkin이라고 합니다.

Then 아래의 And는 앞부분에 같은 종류의 조건을 이어 붙이는 말입니다. 여기서는 설비 명령이 없다는 조건에 업무 처리도 없고 거절로 닫힌다는 결과를 더했어요. 하지 않아야 할 일도 기대 결과로 적는다는 점을 보세요.

Gherkin은 BDD에서 널리 쓰이는 표현 방법입니다. 이 서식을 썼다고 바로 자동 시험이 완성되는 것은 아닙니다. 실행과 대조를 맡을 코드에 연결해야 해요. 여기서는 문장의 구조를 읽고, 이어지는 HYD의 파이썬 시험에서 같은 전제와 행동, 결과가 어떻게 연결되는지 보겠습니다.

[SLIDE 10] Then의 조건: 실행 결과로 가릴 수 있는 사실

Then을 어떻게 적느냐에 따라 시나리오가 쓸모 있느냐 없느냐가 갈립니다. 화면 왼쪽과 오른쪽을 비교해 보세요.

왼쪽은 거절을 적절히 처리한다, 설비에 영향을 주지 않는다, 담당자에게 잘 알린다. 틀린 말은 아니죠. 그런데 실행 결과를 보고 이게 통과인지 실패인지 정할 수 있을까요? 적절하다는 기준이 읽는 사람마다 다르잖아요.

오른쪽은 어떤가요. 설비 명령이 0건이다, 업무 처리가 0건이다, 처리 건이 거절로 닫히고 사유가 남아 있다. 건수를 세거나 상태와 값을 읽으면 바로 가려집니다.

명세의 수용 기준을 적을 때도 판정할 수 있는 문장으로 적었었죠. Then도 똑같습니다.「영향을 주지 않는다」라고 쓰고 싶다면, 그 영향이 무엇으로 나타나는지를 찾아서「설비 명령이 0건이다」로 바꿔 적는 거예요.

명령 0건은 우리가 검사한 명령 생성 경로에 관한 결과입니다. 설비의 모든 물리적 영향이 없었다는 뜻으로 넓혀 읽지는 마세요. 이 시나리오에서 확인할 업무 처리와 발송, 상태 기록의 범위를 함께 정해야 합니다.

[SLIDE 11] SDD와 BDD: 정하는 일과 확인하는 일

명세를 다룬 SDD와 방금 본 BDD를 나란히 놓겠습니다. 왼쪽 SDD는 무엇을 만들고 무엇을 지킬지 명세를 중심으로 정하고, 구현과 검증에도 그 기준을 이어 가는 접근입니다. 오른쪽 BDD는 구체적인 행동 사례를 함께 논의해서 기대를 맞추고, 그 사례를 실행 가능한 검증에 연결하는 접근이에요.

두 방식을 구현 전과 구현 후로 나누면 정확하지 않습니다. 거절하면 명령이 없어야 한다는 시나리오는 구현하기 전에도 쓸 수 있죠. 오히려 그때 사유가 없으면 거절을 받을 것인지 물어 보면, 명세에서 빠뜨린 조건을 찾을 수 있습니다. 구현한 뒤에는 같은 시나리오로 실제 행동을 확인하고요.

이 수업에서는 앞 회차에 명세를 정리했고, 이번 회차에 시나리오와 검증을 다룹니다. 이것은 학습 순서입니다. 실제 작업에서는 명세와 사례를 오가며 기대를 구체화할 수 있어요. 앞 실습에서 저장한 입력과 조회 자료, 기대 결과가 그 출발점입니다.

지키겠다는 문장이 있으면 확인할 사례를 고를 수 있고, 사례가 구체적이면 문장의 모호함을 발견할 수 있습니다. 이렇게 서로 보완하면서 변경의 기준을 남기는 겁니다.

[SLIDE 12] 명세에 적은 것이 시나리오에 놓이는 자리

그럼 명세의 항목 하나하나가 시나리오의 어디로 가는지 표로 맞춰 볼게요. 왼쪽 칸이 명세에 적은 것, 가운데가 시나리오에서 놓이는 자리, 오른쪽이 쿨러 사건의 예입니다.

첫 줄, 입력 계약은 Given과 When에 넣는 입력값이 됩니다. 쿨러 사건이라면 설비 HYD-01과 쿨러 성능 저하 경보가 입력이죠.

둘째 줄, 출력 계약은 Then으로 가요. 결과에 있어야 하는 항목들, 원인이나 추천이나 판단 번호 같은 다섯 항목이 다 있는지를 Then에서 확인합니다.

셋째 줄 금지 조건도 Then으로 갑니다. 다만 이번엔 일어나지 않아야 하는 일로 들어가요. 승인 전에는 설비 명령이 0건이다, 이렇게요. 그래서 이 줄을 강조해 뒀습니다.

마지막 줄 수용 기준은 조금 다릅니다. Then 한 줄이 아니라, 시나리오 한 편 한 편이 통과하는 걸로 확인돼요.「거절하면 명령도 업무 처리도 없다」는 기준은 관련 시나리오의 결과로 확인합니다.

이렇게 놓고 보면 시나리오에 적는 내용은 새로 지어내는 게 아니에요. 명세에 이미 적은 약속을 전제, 행동, 기대 결과로 나누고, 기대 결과는 실행 결과로 가릴 수 있는 사실로 바꿔 옮겨 놓는 겁니다. 약속을 이렇게 적어 두면, 실행 결과를 보고 그 약속이 지켜졌는지를 가릴 수 있어요. 처음에 물었던, 약속을 어떻게 적어 둬야 확인할 수 있느냐의 답이 바로 이겁니다.

항목이 있다는 것뿐 아니라 값의 형식과 뜻도 계약에 맞아야 합니다. 한 수용 기준을 여러 상황에서 확인할 수도 있어요. 거절 한 번의 통과가 모든 입력과 예외를 대신하지는 않습니다.

[SLIDE 13] 실행 가능한 검증과 네 종류의 시험

약속을 시나리오로 적는 방법까지 찾았으니, 두 번째 단원에서는 그 시나리오를 실행 가능한 검증으로 바꾸고, 시험을 네 종류로 나눠 봅니다. Given, When, Then으로 적은 시나리오도 아직은 글이에요. 글로만 두면 결국 사람이 읽고 손으로 확인해야 하죠. 바꿀 때마다 다시 눌러 봐야 하는 반복의 부담이 그대로 남는 겁니다.

그래서 물음이 두 개예요. 시나리오를 어떻게 바꿔야 실행으로 되풀이할 수 있을까요? 그리고 어떤 관점으로 경우를 골라야 중요한 상황을 덜 놓칠까요? 첫 번째 물음부터 풀어 보겠습니다.

[SLIDE 14] 시나리오에서 실행 가능한 검증으로

시나리오를 글로 적었으면 실제 시스템과 연결해야 합니다. 그림 왼쪽의 세 줄이 오른쪽에서 각각 무엇이 되는지 보세요. Given은 시험을 시작할 상태와 입력을 준비하는 코드가 됩니다. When은 그 상태에서 거절 같은 행동을 실행하는 코드가 되고, Then은 나온 결과가 기대 조건을 만족하는지 검사하는 코드가 됩니다.

문장을 적는 것만으로 시스템이 자동 실행되지는 않습니다. 사람이 확인할 수도 있고, 반복할 부분을 시험 코드나 실행 도구에 연결할 수도 있어요. Gherkin을 사용하는 도구에서는 문장에 대응하는 실행 코드를 연결합니다. 다음 장의 HYD 예는 같은 구조를 파이썬 시험으로 직접 작성한 것입니다.

반복해서 비교하려면 입력뿐 아니라 조회 자료와 규칙, 설정도 확인해야 합니다. 같은 경보를 넣어도 센서 값이 바뀌면 다른 결과가 올바를 수 있잖아요. 저장한 자료로 조건을 고정하면 코드 변경의 영향을 비교하기 쉬워집니다. 실시간 자료를 쓰는 검증이라면 그때 읽은 값과 시각을 함께 남겨야 하고요.

HYD의 규칙 판단 경로는 같은 사실과 같은 규칙에서 같은 판정을 내도록 구성돼 있습니다. 그렇다고 자료를 고정하면 모든 AI의 설명까지 같아진다는 뜻은 아니에요. 오른쪽 대조에서는 그 상황

에 맞는 값이나 허용 범위, 일어나지 않아야 할 행동을 검사합니다. 어긋나면 어떤 시나리오의 어느 조건이 실패했는지 남겨 다음 수정으로 이어 갑니다.

[SLIDE 15] 단언: 기대값과 실제 결과를 대어 보는 한 줄

이건 HYD에서 승인 단계를 확인하는 실제 시험 코드 한 편입니다. 코드를 다 읽을 필요는 없어요. 위에서부터 세 군데만 짚을게요.

맨 위의 긴 이름, 이게 시험의 이름입니다. 풀어서 읽으면 「거절은 설비와 업무에 아무 일도 하지 않고, 처리 건을 거절로 닫는다」예요. 시험 이름이 그대로 시나리오 문장인 거죠.

가운데 거절을 부르는 줄이 When입니다. 사유로 「이번 주 생산 일정상 진행하지 않음」을 넣고 거절을 실행하고요.

아래쪽 assert 두 줄이 Then이에요. assert는 단언이라고 하는데, 뒤에 적은 식이 참인지 확인하는 한 줄입니다. 첫 줄은 명령 번호가 없고 업무 처리 목록이 비어 있는지를 보고요. 둘째 줄은 승인 값이 거절을 뜻하는 「반려」인지, 사유가 그대로 남았는지를 봅니다.

화면의 줄임표는 준비 부분하고 나머지 단언을 줄여 놓은 자리예요. 실제 시험엔 메일이 나가지 않았는지 같은 조건도 같이 들어 있습니다. 그리고 단언이 하나라도 참이 아니면, 이 시험은 실패로 기록돼요. 시나리오의 기대가 시험 코드의 검사 조건으로 연결된 겁니다.

이 시험은 준비한 상태와 실행 대역을 사용합니다. 실제 공장에 명령을 보내 본 결과와는 구별해야 해요. 또한 기대 결과 한 줄이 반드시 단언 한 줄과 일대일로 대응하는 것은 아닙니다. 필요한 조건을 빠뜨리지 않고 확인하는지가 중요합니다.

[SLIDE 16] 시험 이름으로 읽는 시나리오

시험 이름을 보면 무엇을 확인하려는지 알 수 있습니다. 영어 이름을 그대로 읽을 필요는 없어요. 왼쪽 상황과 오른쪽 기대 결과를 연결해 보세요.

첫 줄은 권한 있는 승인을 받아 정해진 실행 조건에서 끝까지 처리하는 경우입니다. 둘째는 사유를 적고 거절했을 때 설비 명령과 업무 처리가 없고 거절로 끝나는 경우예요. 셋째는 사유 없는 거절을 받지 않는 경우, 넷째는 지연 알림 뒤에도 승인할 수 있는 경우입니다.

저장소의 승인과 거절 시험은 냉각 대응, 정기 정비, 예비품 구매라는 세 흐름에 같은 시험을 적용합니다. 세 업무가 같은 승인 단계를 공유하기 때문이에요. 그렇다고 모든 시험이 세 흐름에서 똑같이 실행된다는 뜻은 아닙니다. 각 업무의 실제 조치와 별도 조건도 확인해야 하죠.

실패한 시험 이름은 조사할 출발점을 알려 줍니다. 이름만으로 원인이 확정되는 것은 아니므로, 그 시험의 입력과 실패한 단언을 함께 읽어야 합니다.

[SLIDE 17] 한 시나리오가 확인하는 것은 한 경우이다

승인 한 경우가 통과했다고 승인 단계 전체가 확인된 것은 아닙니다. 그림은 같은 출발점에서 갈리는 네 가지 대표 상황이에요.

승인하면 현재 실행 조건도 확인한 뒤 해당 업무가 진행됩니다. 거절하면 설비 명령과 업무 조치 없이 처리 건을 거절로 닫습니다. 응답이 없으면 지연 알림 뒤에도 기다리고, 사유나 권한이 부족한 요청은 받아들이지 않습니다.

이 중 하나만 실행하면 나머지 갈래는 확인하지 못하죠. 그래서 각 행동의 입력과 기대를 준비합니다. 하나의 시험 틀에 여러 입력을 넣어 확인할 수도 있지만, 어떤 경우를 실제로 실행했는지는 구별해서 남겨야 합니다.

이 네 갈래가 가능한 상황을 전부 열거한 것은 아닙니다. 필요한 경우를 빠뜨리지 않으려면 기준값 근처와 자료 누락, 변경의 영향까지 다른 관점으로 살펴봐야 해요. 다음 표의 네 종류가 그 관점을 제공합니다.

[SLIDE 18] 네 종류의 시험: 정상, 경계, 예외, 회귀

중요한 갈래를 고르는 데 쓸 만한 구분이 있습니다. 시험을 확인하는 대상에 따라 정상, 경계, 예외, 회귀, 이렇게 네 종류로 나누는 거예요. 화면 표의 왼쪽 칸입니다.

첫 줄 정상 시험은 기본 경로가 끝까지 가는지를 봅니다. 오른쪽 칸의 예처럼, 승인과 실행 조건을 충족하면 처리 건이 끝까지 처리되는 거죠.

경계 시험은 기준값 바로 안팎에서 판정이 맞는지를 봐요. 쿨러 사건에서는 조치 뒤에 유온을 다시 읽어서 회복했는지를 판정하는데, 이 실습 설비에 정해 둔 회복 기준이 55 ℃ 미만입니다. 그러니 다시 읽은 유온이 딱 55.0 ℃라면 회복으로 보지 않아야 하죠. 이게 경계 시험의 예입니다.

예외 시험은 거절, 누락, 오류처럼 일이 정상 경로를 벗어났을 때 정한 대로 다루는지를 봅니다. 사유 없는 거절이 받아들여지지 않는 게 예외 시험이에요.

마지막 회귀 시험은 무언가를 바꾼 뒤에도 기존 행동이 그대로인지를 봅니다. 긴급 조건을 더한 뒤에도, 거절하면 명령이 0건이어야 하죠.

그런데 넷이 같은 결의 구분은 아니에요. 앞의 셋은 무엇을 확인하느냐에 따른 구분이고, 회귀는 언제 확인하느냐에 가깝습니다. 그래서 회귀 시험의 내용은 대개 이미 있던 정상, 경계, 예외 시험이에요. 정상 시험은 앞에서 본 승인 시나리오가 바로 그 예니까, 나머지 셋을 차례로 보겠습니다.

[SLIDE 19] 경계값 분석: 기준값의 바로 안쪽과 바깥쪽

경계 시험은 판정이 갈리는 지점을 집중해서 확인합니다. 그림의 회복 기준은 유온 55도 미만이에요. 55도보다 낮아야 충족하고, 정확히 55도이면 충족하지 않습니다.

왼쪽의 실제 사건 값은 약 52도였죠. 그 값은 회복 구간 안에 있습니다. 하지만 그 한 번으로 기준값 바로 옆에서도 판정이 맞는지는 알 수 없어요. 미만을 이하로 잘못 구현했어도, 구간 안쪽의 값에서는 같은 결과가 나오니까요.

그래서 여기서는 소수 첫째 자리까지 값을 넣는 예로, 54.9도와 정확히 55도, 55.1도를 비교합니다. 첫 값만 회복 기준을 충족해야 해요. 기준 바로 아래와 기준 자체를 비교하면 등호를 잘못 넣은 오류를 찾을 수 있고, 위쪽 값으로 반대 구간도 확인할 수 있습니다.

0.1도 간격이 모든 센서의 공통 기준은 아닙니다. 실제 자료의 정밀도와 시스템이 값을 다루는 단위에 맞춰 시험 값을 고릅니다. 예상과 다른 결과가 나오면 비교 연산뿐 아니라 단위 변환이나 반올림, 읽은 값이 맞는지도 살펴봐야 해요. 기준에서 먼 값 하나보다 경계를 나누어 보는 이유입니다.

[SLIDE 20] 시간의 경계: 승인 지연 시간의 직전과 직후

숫자의 경계는 시간에도 있습니다. HYD의 이 승인 흐름은 업무상 10분을 지연 알림 기준으로 씁니다. 화면은 그 직전, 기준을 지난 뒤, 알림 뒤의 대기, 뒤늦은 승인을 나눈 것입니다.

먼저 기한 직전에는 지연 알림이 없어야 합니다. 기한에 도달해 확인하는 시점에는 알림 대상이 되고요. 자동 시험에서는 시간을 진행시켜 기한 전후의 상태를 확인할 수 있습니다. 실제로 10분 동안 기다려야만 시험할 수 있는 것은 아니에요. 수업 환경의 20배속에서는 업무상 10분이 실제 30초에 해당합니다.

다음은 알림 이후입니다. HYD의 이 정책에서는 응답이 늦었다고 자동 거절하거나 승인하지 않습니다. 지연을 알리고 처리 건을 대기 상태로 유지해요. 저장소 시험에는 훨씬 뒤에도 처리 건이 계속 대기하는지 확인하는 경우가 있습니다.

마지막으로 뒤늦은 승인도 접수할 수 있어야 합니다. 다만 승인을 받았다는 것과 조치를 실행할 수 있다는 것은 다르죠. 현재 조건 재검사도 통과해야 실행으로 이어집니다. 다른 업무에서는 기한 뒤 자동 종료 정책을 둘 수도 있지만, 이 시험이 확인하는 기준은 HYD에 정해 둔 대기 정책입니다.

[SLIDE 21] 예외 시험: 잘못되었을 때 정한 대로 행동하는가

예외 시험은 잘못된 입력이나 실패가 생겼을 때 정한 대로 대응하는지 봅니다. 화면의 네 카드는 HYD에서 살펴볼 대표 상황입니다.

도구 오류가 나면 오류를 기록하고, 읽지 못한 값을 만들어 넣지 않아야 합니다. 근거가 오래됐다면 현재 판단에 쓸 수 있는지 확인하고 부족하면 보류해야 해요. 여기의 60초는 HYD 사례의 신선도 기준입니다. 방금 조회했다는 이유만으로 오래된 관측값이 최신 값이 되지는 않습니다.

필수 정보가 없으면 무엇이 부족한지 구별합니다. 사람의 답으로 보완하는 흐름에서는 질문하고, 근거 부족으로 보류하는 흐름에서는 사유를 남깁니다. 질문을 했다고 승인 권한까지 생기는 것은 아니죠.

권한 없는 요청은 거부돼야 합니다. HYD는 실제 역할 구성원인지와 지정된 승인 역할 또는 더 높은 등급인지 함께 확인합니다. 역할 이름을 직접 적거나 같은 등급이라는 이유만으로 다른 역할의 권한을 얻지는 않습니다.

여기서 기대한 오류나 보류가 나왔다면 시험은 통과할 수 있어요. 업무가 정상 완료되지 않았다는 것과 시험이 실패했다는 것은 다릅니다. 시험은 그 상황에서 약속한 대응이 나왔는지를 확인하니까요.

[SLIDE 22] 회귀 시험: 변경 뒤에 다시 실행하는 기존 시나리오

마지막이 회귀 시험입니다. 무언가를 바꾼 뒤에 기존 시나리오를 다시 실행하는 시험이에요.

화면 왼쪽이 변경 전입니다. 앞에서 본 승인 단계의 네 갈래, 승인, 거절, 무응답, 사유 없는 요청의 시나리오 넷이 모두 통과한 상태죠. 가운데 화살표에서 요구가 바뀌어요. 명세에 긴급 조건이 하나 더해집니다.

오른쪽 변경 후를 보면, 맨 아래에 새 시나리오가 하나 늘었죠. 긴급 조건을 확인하는 시나리오입니다. 이때 새 시나리오만 돌려 보고 통과했다고 끝내면 어떨까요? 긴급 조건이 잘 동작하는 건 알지만, 거절이나 무응답의 행동이 그대로인지는 모릅니다.

그래서 파란 괄호처럼 기존 넷도 전부 다시 실행해요. 이게 회귀 시험입니다. 회귀라는 말은 되돌아간다는 뜻이에요. 변경으로 기존 행동에 의도하지 않은 영향이 생겼는지 찾는 시험입니다. 바꾼 곳만 확인해서는 이걸 잡을 수 없어요.

[SLIDE 23] HYD의 회귀 시험 기록

화면은 저장소에 남아 있는 서로 다른 시점의 검증 기록입니다. 이번 교재 수정에서 이 시험 전체를 새로 실행했다는 뜻은 아니에요. 기록을 읽는 방법을 보겠습니다.

왼쪽 전체 시험은 2,673개 통과, 16개 건너뜀, 실패 0건으로 적혀 있습니다. 건너뛴 항목은 통과한 항목과 구별해야 하죠. 가운데 쿨러 사건 기록은 경보가 처리 건을 열고, 권한 없는 승인이 거부되고, 승인 뒤 명령과 재관측, 후속 처리가 이어지는지 등 40항목을 확인했습니다. 오른쪽 펌프와 팬 사건은 별도 기록에서 43항목을 확인했습니다.

숫자는 그때 실행한 범위의 요약입니다. 통과 수가 줄었다고 곧바로 기능이 깨졌다는 뜻은 아니에요. 시험 목록과 환경이 달라졌거나 건너뛴 항목이 늘었을 수도 있습니다. 실행 시점과 버전, 실패하거나 건너뛴 이유를 함께 확인해야 합니다.

기존 시험을 남겨 두는 가치는 다음 변경에서 같은 기대를 다시 확인할 수 있다는 데 있습니다. 다만 지금의 요구와 환경에 맞는지도 관리해야 하고, 시험에 없던 상황까지 모두 확인했다고 말할 수는 없습니다.

[SLIDE 24] 에이전트 출력의 반복 검증

세 번째 단원은 에이전트 출력의 반복 검증입니다. 앞에서는 거절하면 명령이 없는지처럼 정해진 조건을 확인했습니다. 에이전트에서도 같은 원칙을 쓰지만, 설명 문장이나 실행 과정은 매번 같지 않을 수 있어요.

그렇다면 무엇을 같은 것으로 비교하고, 어떤 차이를 실패로 세어야 할까요? 실제 기록을 먼저 보고 기준을 나눈 뒤, 여러 실행을 함께 해석하는 방법으로 이어가겠습니다.

[SLIDE 25] 같은 종류의 경보를 처리한 두 실행의 비교

이번에는 같은 종류의 쿨러 경보를 처리한 두 기록을 비교해 보겠습니다. 왼쪽은 두 기록에서 같았던 내용이에요. 원인은 쿨러 핀 오염, 추천은 팬 최대 운전이었고, 조치 후보의 평가 점수도 같았습니다.

오른쪽은 달랐던 부분입니다. 진행을 설명한 문장과 도구 호출의 흐름, 걸린 시간이 달랐어요. 한 기록에는 업무 자료 조회와 조치 효과 예측이 나타납니다. 다만 두 경보는 서로 다른 처리 건이고, 한쪽 기록의 전체 도구 호출을 모두 대조한 것은 아닙니다. 모든 입력을 고정한 반복 실험이라고 읽으면 안 돼요.

여기서 구별할 것은 판단 도구의 계산과 에이전트가 일을 진행하는 과정입니다. HYD의 규칙 판단 도구는 같은 사실과 같은 규칙을 주면 같은 판정을 내는 결정론적 경로를 제공합니다. 에이전트는 자료를 찾아 설명하는 과정에서 다른 선택을 할 수 있고요. 실시간 값이 달라지면 규칙 도구의 입력도 달라집니다.

따라서 두 기록의 차이를 모두 모델의 무작위성 때문이라고 결론 내리지는 않습니다. 대신 이 비교를 출발점으로 삼아, 다음 장에서 무엇이 달라도 되는지와 무엇은 반드시 지켜야 하는지를 정해 보겠습니다.

[SLIDE 26] 표현 차이와 계약 위반

시험을 만들기 전에 달라도 되는 것과 지켜야 하는 것을 가릅니다. 화면 왼쪽의 설명 문장은 같은 사실을 다른 낱말로 전달할 수 있어요. 글자가 다르다는 이유만으로 실패로 세면, 의미가 맞는 결과까지 탈락합니다. 대신 근거와 결론이 맞는지는 확인해야 하죠.

도구 순서와 걸린 시간은 조금 더 주의해서 봅니다. 서로 독립적인 조회의 순서는 바뀔 수 있지만, 승인 전에 실행하거나 오래된 값을 최신 값처럼 쓰면 안 됩니다. 처리 시간에 기준이 있다면 그 기준도 지켜야 하고요. 과정의 차이는 필요한 의존 관계와 권한, 시간 조건을 충족하는 범위에서 허용합니다.

오른쪽은 계약 위반입니다. 필수 결과 항목이 빠지거나, 승인 전에 설비 명령과 후속 업무 조치를 실행하거나, 요구한 근거가 없거나, 금지한 조치를 추천하는 경우예요. 이런 위반은 한 번이라도 발견되면 따로 실패로 남깁니다. 판단을 제출하거나 진행 기록을 남기는 모든 쓰기를 승인 위반으로 세는 것은 아닙니다. 금지한 행동의 범위를 정확히 적어야 해요.

너무 엄격하게 글자를 대조하면 의미 있는 차이를 놓치고, 너무 느슨하게 비슷해 보인다고 넘기면 계약 위반을 놓칩니다. 그래서 형식과 의미, 실제 행동의 조건을 나누어 검사합니다. 명세에서 빠진 중요한 조건을 발견했다면 그냥 자유롭게 둔 것이 아니라 기준을 보완할 문제로 다뤄야 합니다.

[SLIDE 27] Then이 견주는 것: 문장이 아니라 계약이 정한 항목

이 구분을 실제 출력에 대어 보겠습니다. 화면의 긴 식별자는 철자를 외우기 위한 것이 아니에요. 어떤 항목을 어떤 방식으로 확인하는지 보시면 됩니다.

첫째는 다섯 필수 항목입니다. 원인과 고장 유형, 진단 가이드 카드, 판단, 판단 번호가 있어야 하고, 약속한 형식과 뜻도 맞아야 합니다. 이름만 있는 빈 값으로는 충분하지 않죠.

둘째는 원인과 추천 조치입니다. 예시에서는 쿨러 핀 오염과 팬 최대 운전을 가리킵니다. 정답표가 하나의 원인을 정했다면 그 값과 대조하고, 여러 조치를 허용했다면 추천이 허용 목록 안에 있는지 확인합니다. 실행마다 같은 글자를 요구할지, 허용한 범위를 검사할지는 기대 행동에 따라 정하는 겁니다.

셋째는 설명입니다. 문장을 통째로 복사했는지를 묻지는 않지만, 선택한 원인과 조치를 근거에 맞게 설명했는지는 봐야 합니다. 인용 번호가 있다고 그 설명까지 자동으로 옳아지는 것은 아니에요.

넷째는 판단 번호입니다. 제출마다 새로 생기므로 이전 번호와 같아야 하는 값은 아닙니다. 번호가 있고 형식이 맞는지에 더해, 실제로 저장된 해당 판단을 가리키는지도 확인할 수 있어야 합니다.

HYD의 실행 중 출력 검사는 필수 항목과 형식을 확인하고, 잘못되면 정해진 횟수 안에서 수정을 요청합니다. 이 형식 검사와 지금 설명한 의미 검증은 범위가 다릅니다. 형식 통과를 판단의 정답으로 읽지 않기 위해 다음 장의 채점을 따로 보는 거예요.

[SLIDE 28] 판단 채점: 정답표와 견주는 네 항목

형식이 맞아도 원인을 잘못 고를 수 있습니다. HYD의 판단 채점은 사람이 작성한 정답표와 결과를 대조해서 그 차이를 확인합니다. 조치 후보의 우선순위를 매기는 평가 점수와도 구별해 주세요. 여기서는 결과가 기대에 얼마나 맞았는지를 채점합니다.

표의 첫 줄은 원인과 고장 유형입니다. 두 값이 모두 기대와 맞아야 이 항목을 충족해요. 둘째 줄은 1순위 추천이 허용한 조치 중 하나인지 봅니다. 셋째 줄은 금지 조치를 확인하는데, 후보에서 제외해야 하는지, 1순위로 추천하지 않으면 되는지를 정답표에서 구별합니다. 두 조건은 범위가 다르죠.

넷째 줄은 요구한 근거 식별자가 결과에 얼마나 들어 있는지입니다. 이것은 근거 포함률이에요. 인용한 자료가 설명을 실제로 뒷받침하는지까지 자동 증명하는 점수는 아닙니다.

오른쪽의 30, 30, 20, 20은 HYD의 기본 가중치입니다. 원인과 고장 유형이 틀리면 첫 항목의 30점을 받지 못하고, 일부 금지 조건이나 근거만 충족하면 해당 항목에는 비율에 따른 점수가 붙습니다. 전체는 100점 만점이지만, 이 숫자가 정답일 확률을 뜻하지는 않아요.

정답표는 처음부터 모든 답이 들어 있는 자료가 아닙니다. 해당 상황의 기대 원인과 허용 조치, 금지 조건과 근거를 사람이 검토해 넣어야 합니다. 설명 문장의 품질은 이 네 항목 점수에 직접 들어가지 않으므로 별도로 검토합니다. 채점이 확인하는 범위를 알고 써야 점수가 높은데도 잘못된 설명을 놓치는 일을 줄일 수 있습니다.

[SLIDE 29] 반복 실행: 한 번의 통과로는 모자란 까닭

한 번 좋은 결과가 나와도 다른 실행에서도 같은 기준을 지키는지는 남아 있습니다. 같은 평가 항목을 여러 번 실행하고, 화면의 평균과 최저, 결과의 반복 비율을 함께 보겠습니다.

평균은 이번에 관찰한 실행들의 전반적인 점수입니다. 최저는 그중 가장 낮았던 점수예요. 평균이 높아도 한 번 크게 틀렸다면 최저에서 드러납니다. 다만 관찰한 최저가 앞으로 가능한 모든 경우의 최악을 뜻하지는 않습니다.

세 번째는 HYD 화면에서 흔들림이라고 부르는 값입니다. 코드에서는 원인과 추천 조치의 조합 중 가장 자주 나온 조합의 비율을 계산합니다. 값이 클수록 그 조합이 일관됐다는 뜻이에요. 100퍼센트여도 전체 설명과 모든 근거가 같다는 뜻은 아니고, 같은 오답을 반복했을 수도 있습니다. 일관성과 정답 여부를 함께 봐야 하는 이유죠.

필수 항목 누락이나 승인 전 조치 실행 같은 위반은 평균 점수에 섞어 넘기지 않습니다. 발생 여부와 횟수, 해당 실행의 기록을 따로 남깁니다. 다른 실행의 높은 점수가 그 위반을 없애 주지는 않으니까요.

이 도구는 반복 횟수를 최대 열 번으로 제한하지만, 열 번이면 충분하다는 일반 기준은 아닙니다. 규칙만으로 같은 조건을 계산하는 경로는 똑같은 입력의 단순 반복보다 경계값과 다른 상황을 확인하는 편이 유용합니다. 모델 경로를 비교할 때도 입력과 조회 자료, 설정을 기록해서 무엇이 바뀌었는지 구별해야 합니다.

[SLIDE 30] 실패한 시험에서 고칠 곳 가리기

실패를 찾았다면 이제 원인을 좁혀야 합니다. 표의 가운데 칸은 정답처럼 고칠 곳을 지정하는 칸이 아니라, 먼저 대조할 자료입니다.

단계를 건너뛰었다면 지침과 호출 기록을 함께 봅니다. 지침이 모호했는지, 필요한 도구를 쓰지 않았는지, 호출했지만 실패했는지에 따라 조치가 달라져요. 기준값이 다르면 규칙과 그 버전, 실제 입력과 단위를 확인합니다.

결과 항목이 빠졌다면 먼저 명세에 그 항목이 요구돼 있는지 봅니다. 이미 요구돼 있다면 출력 구현이나 검사가 놓친 것일 수 있어요. 누락됐다는 이유만으로 명세에서 그 항목을 지우는 것은 해결이 아닙니다.

반대로 시험의 기대값을 옮겨 적다가 잘못 적었을 수도 있습니다. 그때는 원래 요구와 근거를 대조해서 시험을 바로잡고 이유를 남깁니다. 요구가 바뀌었다면 명세와 시나리오를 함께 갱신해야 하고요.

마지막 줄이 가리키는 것은 근거 없이 기대값을 낮추는 일입니다. 현재 나온 결과를 통과시키려고 기준을 바꾸면 같은 오류가 다음에도 통과합니다. 실패 기록에서 입력, 기대, 실제 결과를 확인한 다음 원인을 고치고, 해당 시험과 기존 시험을 다시 실행하겠습니다.

[SLIDE 31~32] 실습: 새 기능과 기존 기능의 행동 기준 작성

지금까지 세 단원에 걸쳐, 명세의 약속을 Given, When, Then으로 적고, 그 시나리오를 시험으로 바꾸고, 실행마다 달라지는 출력에서 무엇을 실패로 셀지까지 가려 봤죠. 이제 그걸 직접 해 볼 차례입니다. 실습은 넷이 이어지는데요, 첫 실습의 할 일은 명세 spec.md를 썼던 그 에이전트를 그대로 가져와서, 다섯 경우의 행동 기준을 시나리오 문서로 적는 거예요.

화면 둘째 카드가 그 다섯 경우예요. 일반 정비, 긴급 정비, 필수 정보 누락, 승인 거절, 도구 오류. 경우마다 Given, When, Then을 나눠 적고, 입력값하고 기대 상태, 지켜야 할 출력 계약도 같이 적습니다.

다 적었으면 마지막 카드의 성공 기준으로 두 가지를 점검하겠습니다. Then이 건수나 값처럼 실행 결과로 가릴 수 있는 사실인지, 그리고 금지 조건이 일어나지 않아야 할 일로 들어 있는지요. 이 실습이 끝나면 그 다섯 경우의 시나리오 문서가 남습니다.

이어서 안내할 두 번째 실습에서는 이 문서를 실제 실행과 연결합니다. 입력값뿐 아니라 비교할 조회 자료와 기대 결과도 함께 남기겠습니다.

[SLIDE 33~34] 실습: 에이전트 동작의 반복 검증

두 번째 실습에서는 첫 실습에서 작성할 다섯 경우의 시나리오로, 에이전트의 동작을 반복 검증하는 실습을 준비합니다. 적어 둔 시나리오는 아직 글이라, 에이전트가 정말 그렇게 움직이는지는 돌려 봐야 알죠. 화면 첫 번째 카드의 입력이 그 다섯 경우의 시나리오 문서예요. 이걸 준비된 자동 시험이나 반복 실행 스크립트에 잇고, 같은 입력으로 여러 번 실행합니다.

실행 결과는 로그와 결과 파일로 남아요. 실제 출력과 기대값을 견줘서, 어느 조건에서 규칙을 어겼는지 찾아 위반 목록에 적겠습니다.

이 실습의 성공 기준은 어긋남마다 판정이 붙어 있는 겁니다. 설명 문장이 달라진 표현 차이인지, 항목이 빠지거나 금지된 일이 일어난 계약 위반인지를 가르면 돼요. 실습이 끝나면 실행 결과 로그와, 판정이 붙은 위반 목록이 남습니다.

조회 자료와 설정도 함께 기록해 차이가 생긴 조건을 구별하겠습니다. 허용할 과정의 차이인지, 실제 조건을 어긴 것인지 근거를 붙이는 것이 핵심입니다.

[SLIDE 35~36] 실습: 오류 수정과 전체 회귀 검증

세 번째 실습에서는 반복 검증에서 남길 위반 목록을 출발점으로, 오류를 고치고 기존 기능이 그대로인지 확인하는 실습을 준비합니다. 위반 목록에서 실제로 실패한 시나리오만 골라 원인을 따라갑니다.

실패의 증상을 보고, 고칠 곳이 프롬프트 같은 지침인지, 지식 그래프의 기준값인지, 명세인지, 코드인지부터 가려 보겠습니다. 이론에서 본 고칠 곳의 표 그대로예요. 가린 곳을 고친 다음 그 시나리오를 다시 돌리고요. 이때 기대값을 낮춰서 통과시키지는 않습니다.

마지막에는 전체 회귀를 돌립니다. 긴급 조건이 동작하면서, 기존의 일반 요청, 필수 정보가 빠진 요청을 확정하지 않는 보류, 그리고 거절 조건이 그대로 통과하면 성공이에요. 무엇을 왜 고쳤는지 적은 수정 기록과 전체 회귀 결과가 남습니다.

실패가 없다면 억지로 오류를 만들거나 고칠 필요는 없습니다. 확인한 범위와 남은 경우를 기록하겠습니다. 실패가 있다면 수정한 시험뿐 아니라 준비한 전체 시나리오를 다시 확인합니다.

[SLIDE 37~38] 실습: 업무 데이터를 연결한 교대 인수인계서 작성

네 번째 실습은 여러 출처를 연결해 교대 인수인계서를 만드는 활동입니다. 앞의 세 실습이 기대를 적고 실행해서 고치는 과정이라면, 여기서는 결과의 바탕이 되는 업무 자료를 어떻게 함께 읽을지 확인합니다. 다음 담당자가 알아야 할 완료 작업과 남은 요청, 주의 사항을 세 출처에서 정리할 거예요.

입력은 설비 이벤트, 정비 이력, 부품 자료의 조회 결과입니다. 실습 환경에 따라 실제 조회 결과 또는 같은 스키마로 준비한 자료를 사용합니다. 단순히 세 문서를 이어 붙이는 것이 아니라, 같은 설비나 요청을 가리키는지와 각 기록의 시점을 맞춰 봐야 합니다.

두 번째 카드의 데이터 패브릭은 여러 곳의 데이터를 연결하고 관리해 활용하는 더 넓은 접근입니다. 데이터를 옮기지 않는 방식만을 뜻하지는 않아요. 이번 실습에서는 그중 원천의 자료를 연결해 읽고 출처를 추적하는 관점을 다룹니다. 새 데이터 플랫폼을 구축하는 활동은 아닙니다.

인수인계 항목마다 어느 자료를 근거로 썼는지와 그 자료의 시각을 붙이겠습니다. 방금 조회한 과거 기록일 수도 있으므로, 조회 시각과 사건이 일어난 시각을 구별해야 합니다. 한 출처를 읽지 못했다면 관련 작업이 없다고 단정하지 말고 확인하지 못한 범위를 적습니다. 출처끼리 상태가 다르면 어느 시점의 기록인지 확인하고, 해결되지 않은 차이도 남겨야 해요.

성공 기준은 다음 담당자가 각 항목의 근거와 미확인 부분을 찾아갈 수 있는 것입니다. 네 실습의 목표를 모두 살펴봤습니다. 이제 시나리오 작성부터 반복 검증과 수정, 교대 인수인계 순서로 직접 진행하겠습니다.

[이론 06] 에이전트 제작부터 검증까지 전체 과정 정리하기 — Wrap-up

[SLIDE 39] 에이전트 제작부터 검증까지의 전체 과정

이번 시간에는 네 실습에서 남긴 시나리오와 실행 기록, 수정 내역과 인수인계서를 바탕으로 전체 과정을 정리합니다. 실패가 발견됐다면 어떻게 고쳤는지, 아직 확인하지 못한 조건은 무엇인지도 함께 떠올려 보세요.

우리는 목표와 도구로 에이전트의 일을 구성하고, 스킬과 연결 규약으로 수행 방법을 정리했습니다. 사람의 개입과 명세를 통해 지킬 기준을 세웠고, 이번에는 그 기준을 구체적인 상황에서 검증했습니다.

이제 쿨러 사건 하나에 그 역할을 차례로 놓아 보겠습니다. 새 기능을 더 배우기보다, 어느 장치가 무엇을 확인하고 그 결과가 다음 일로 어떻게 이어지는지 정리하는 시간입니다.

[SLIDE 40] 여섯 개념이 차례로 푼 문제

여섯 개념을 각각 어떤 문제에 답했는지로 묶어 보겠습니다. 그림의 왼쪽은 일을 수행하는 구조, 오른쪽은 기대와 권한을 확인하는 구조입니다. 이 과정에서 우리가 함께 사용한 개념들이에요.

에이전트는 목표에 맞춰 도구를 선택하고 결과를 보며 다음 행동을 이어 갑니다. 스킬은 반복해서 쓸 절차와 판단 지침을 묶어 제공합니다. 지침을 적었다고 실행 순서가 항상 강제되는 것은 아니므로 실제 호출 기록도 확인해야 하죠. MCP는 도구와 자료를 연결하는 공통 규약이고, HYD는 이를 이용해 업무 시스템을 연결했습니다.

HITL은 필요한 정보나 결정을 사람에게 받는 자리를 둡니다. SDD는 명세를 구현과 변경의 기준으로 유지하고, BDD는 구체적인 행동 사례로 기대를 맞추고 검증에 연결합니다.

이 여섯 이름을 모든 에이전트가 반드시 같은 순서로 갖춰야 하는 목록으로 외우지는 마세요. 업무에 따라 도구를 연결하는 방식과 검토 절차는 달라질 수 있습니다. 중요한 것은 일을 수행할 수 있는가에 더해, 무엇을 지켜야 하고 누가 결정하며 어떻게 확인할 것인가를 함께 설계하는 일입니다.

[SLIDE 41] 쿨러 사건 한 건 위에 놓인 장치

같은 개념을 쿨러 사건의 흐름에 놓아 보겠습니다. 왼쪽에서 경보가 열리고, 에이전트가 스킬을 읽어 도구를 호출하고, 추천안을 제출합니다. 오른쪽에서는 검사와 담당자의 승인, 현재 조건 확인을 거쳐 설비 명령으로 이어집니다.

기록의 경보 시각은 11시 13분 8초, 승인 시각은 11시 14분 56초였습니다. 이 두 시점 사이는 약 2분이에요. 전체 복구 시간이나 에이전트 혼자 일한 시간을 뜻하는 것은 아닙니다.

도구 호출 과정에서는 대안 평가 응답이 커서, 필요한 항목으로 좁혀 보겠다는 계획도 남겼습니다. 앞에서 확인한 기록만으로 실제 재조회가 성공해 모든 항목을 읽었다고까지 결론 내리지는 않았죠. 계획과 수행 기록을 나눠 읽어야 근거가 끊긴 곳을 찾을 수 있습니다.

추천안의 자동 검사는 정해진 조건을 확인하고, 담당자의 승인은 실행에 대한 결정을 남깁니다. 두 자리를 통과해도 승인 시점의 설비 조건을 다시 확인해야 합니다. 이 그림은 각 역할을 묶은 것이고, 실제 처리에는 그 사이의 검사들이 함께 있습니다.

아래 시나리오와 회귀 시험은 이 흐름을 정한 입력과 조건에서 다시 확인합니다. 사건 한 건의 성공 기록은 그 사례의 근거이고, 여러 조건을 시험하는 일은 별도의 검증입니다. 둘을 함께 봐야 같은 방식으로 맡길 수 있는 범위를 판단할 수 있어요.

[SLIDE 42] 목표, 도구, 멈춤 조건: 하나가 빠졌을 때

이번에는 목표와 도구, 멈춤 조건이 각각 어떤 빈칸을 메우는지 보겠습니다. 없으면 반드시 어떤 오답이 나온다는 예언이 아니라, 설계할 때 확인할 질문입니다.

목표에는 무엇을 내면 일이 끝나는지가 들어갑니다. 쿨러 사건이라면 단순히 설명을 잘하는 것을 넘어, 근거 있는 추천안을 약속한 형태로 제출하는 것이죠. 끝의 기준이 모호하면 불필요한 조회를 이어 가거나 필요한 일을 덜 한 상태에서 끝날 수 있습니다.

도구는 현재 자료를 확인할 통로입니다. 입력으로 받은 자료만으로 충분한 업무도 있지만, 지금의 유온이나 납기 상황이 필요한 업무라면 그 자료를 얻을 경로가 있어야 해요. 도구가 있어도 필요한 값을 실제로 읽었는지 확인해야 합니다.

멈춤 조건은 근거 부족과 도구 실패를 어떻게 다룰지 정합니다. 확인하지 못한 값을 추정해 결과를 완성하지 않고, 질문하거나 보류하고 사유를 남기는 조건이에요. 중단 기록을 보면 업무를 못했다는 사실뿐 아니라 무엇이 부족했는지도 알 수 있습니다.

목표는 끝을, 도구는 근거의 통로를, 멈춤 조건은 확인하지 못했을 때의 행동을 정합니다. 이 세 가지가 이어져야 성공한 결과와 보류한 결과 모두를 해석할 수 있습니다.

[SLIDE 43] 근거가 놓인 자리: 지식과 현재값

에이전트가 추천안을 낼 때 쓴 근거는 어디서 왔을까요? 화면에 두 갈래로 나눠 놨습니다.

위쪽 회색 구획이 지식이에요. 원인 후보 셋에서 고장 유형을 거쳐 조치 방법으로 가는 관계죠. 쿨러 핀 오염, 팬 구동 불량, 주변 온도 상승이 쿨러 냉각 성능 상실을 일으키고, 그 고장 유형에 대한 완화 후보 중 하나가 팬 최대 운전입니다. 원인과 현재 조건에 따라 그 후보를 적용할 수 있는지는 따로 가립니다. 이건 지식 그래프에 미리 정해 둔 관계입니다.

아래쪽 하늘색 구획은 현재값이에요. 냉각 효율이 절반 아래로 떨어졌다는 값은 시계열 DB에서, 납기가 걸린 생산 오더는 업무 DB에서 읽었습니다. 도구를 통해 판단에 사용한 자료예요. 조회한 시각과 값이 관측된 시각도 함께 확인해야 합니다.

그런데 이 두 갈래는 서로가 필요해요. 지식에는 원인마다 어느 센서 값을 어떤 조건으로 확인할지가 적혀 있습니다. 쿨러 핀 오염이라면 냉각 효율이 낮아졌는지를 보라는 식이죠. 하지만 그 값이 지금 몇인지는 지식에 없어요. 판단하는 순간 시계열 DB에서 읽어 와야, 이 원인이 지금 사건에 맞는지 가릴 수 있습니다. 그래서 두 갈래가 가운데 도구 호출에서 모여서, 오른쪽 추천안이 되는 거예요.

지식의 규칙과 관계도 개정될 수 있고, 관측값은 시간에 따라 달라집니다. 적용한 지식 버전과 관측 시점을 함께 남겨야 해요. 둘을 섞지 않는 것이 근거를 다루는 기준이었던 까닭이 여기 있습니다. 추천안에는 근거마다 어디서 읽었는지가 남아 있어요. 덕분에 나중에 결과가 이상하면, 어느 갈래에서 틀렸는지 거슬러 갈 수 있습니다.

[SLIDE 44] 스킬과 MCP: 바꾸려는 것에 따라 고칠 곳

일하는 방법을 바꾸는 것과 연결할 수 있는 범위를 바꾸는 것은 고칠 자리가 다릅니다. 왼쪽 스킬은 어떤 근거부터 확인하고 부족할 때 어떻게 대응할지 같은 지침을 담습니다. 오른쪽 도구 연결은 어느 시스템의 어떤 기능을 사용할 수 있는지와 관련돼요.

예를 들어 같은 자료를 읽되 확인 순서를 개선하려면 스킬을 검토할 수 있습니다. 새 업무 시스템의 자료가 필요하다면 도구 제공과 연결, 접근 권한을 확인해야 하죠. 호출은 됐는데 권한 오류가 났다면 지침 문장을 길게 쓰는 것으로 해결되지는 않습니다.

출력 항목을 바꾸는 경우에는 스킬만 고쳐서도 안 됩니다. 명세의 계약과 결과를 읽는 프로그램, 출력 검사를 함께 확인해야 합니다. 스킬이 약속을 설명하는 자리라면 실제 검사는 구현이 맡기 때문이에요.

MCP는 연결 규약입니다. 그 이름 자체가 모든 권한을 통제하거나 실행 환경 전체를 격리한다는 뜻은 아닙니다. 실제 허용 범위는 호스트와 서버의 설정, 계정 권한과 도구 구현이 결정합니다. 그래서 원하는 일을 못 했을 때도 지침과 연결, 실제 응답을 구별해서 살펴보는 겁니다.

[SLIDE 45] 검사를 통과한 것과 승인된 것

자동 검사를 통과한 것과 사람이 승인한 것을 구별해 보겠습니다. 가운데의 자동 검사는 HYD 추천안의 형식과 규칙 인용, 승인 역할 같은 조건을 확인합니다. 통과는 정해진 검사에서 위반이 발견되지 않았다는 뜻이에요.

오른쪽 담당자의 승인은 그 안을 실행하도록 허락한 결정입니다. 승인 권한을 확인한 경로에서 사람과 역할, 선택한 안과 시각을 기록합니다. 에이전트가 승인자 이름을 결과에 써 넣었다고 이 기록이 만들어지는 것은 아닙니다.

앞 실습의 다른 AI가 초안을 검토한 것도 떠올려 보세요. 그 검토는 참고할 의견입니다. 자동 검사 통과나 사람의 승인을 대신하지 않아요. 여러 AI가 같은 답을 했더라도 근거와 권한을 따로 확인해야 합니다.

그림 아래의 질문과 작업 닫기는 또 다른 사람의 개입입니다. 부족한 정보를 답하는 것과 실행을 허락하는 것은 역할이 다르죠. 승인 뒤에도 현재 조건을 다시 검사하고, 명령을 보낸 뒤에는 실행 결과를 확인해야 합니다. 그림의 실행 허락을 실제 조치의 성공과 같은 뜻으로 읽지 않는 것이 중요합니다.

[SLIDE 46] 명세, 시나리오, 가드레일, 승인이 추천안에 걸리는 자리

명세와 시나리오, 가드레일과 승인은 각각 어디에 걸릴까요? 한 그림에 이어 봤습니다. 왼쪽 위 명세에서 출발할게요.

명세에서 화살표가 두 개 나가죠. 하나는 가운데 에이전트로 가서, 에이전트가 지킬 기준이 됩니다. 다른 하나는 아래 시나리오로 가서 기대값이 되고요. 같은 명세가 만드는 쪽에는 기준이, 확인하는 쪽에는 기대값이 되는 겁니다. 출력 계약으로 예를 들어 볼게요. 원인부터 판단 번호까지 다섯 항목을 낸다는 약속은, 에이전트 쪽에서는 작업에 붙은 결과 형식이 되고, 시나리오 쪽에서는 「다섯 항목이 있고 원인과 추천이 이 식별자다」라는 Then이 됩니다.

왼쪽 아래 시나리오에서는 에이전트로 화살표가 가요. 에이전트의 행동과 그 결과를 이어 받는 업무 흐름을 확인하는 거죠.

에이전트는 가운데 추천안을 냅니다. 그 추천안에 오른쪽 위의 가드레일이 검사를 걸고, 오른쪽 아래의 담당자가 승인을 걸어요.

여기서 화살표가 닿는 곳을 보세요. 시나리오는 에이전트와 연결된 업무 흐름을 검증하고, 가드레일과 승인은 그 에이전트가 낸 결과 하나하나에 닿습니다.

[SLIDE 47] 요구가 바뀔 때의 순서

명세와 시나리오와 회귀는 변경 한 번 안에서 어떤 순서로 쓰일까요? 화면에 네 단계로 이어 놨습니다. 단계마다 아래 줄은 앞의 실습에서 다룬 긴급 정비 조건의 예예요.

요구가 바뀌면 코드부터 고치지 않습니다. 첫 단계는 명세예요. 새 조건의 계약과 금지 조건, 수용 기준을 적습니다. 긴급 조건을 더하더라도, 「승인 없이 확정하지 않는다」는 그대로 남겨 두고요. 왜 이걸 굳이 다시 적을까요? 긴급이라는 조건은 빨리 처리하자는 쪽으로 기울기 쉽죠. 모호하게 덧붙인 기능이 기존 출력이나 승인 규칙을 흔들 수 있다는 건 명세를 배울 때 봤습니다. 지킬 것을 명세에 그대로 남겨 둬야, 나중에 전체를 다시 돌릴 때 그 약속이 아직 지켜지는지를 확인할 기준이 생깁니다.

둘째 단계에서 그 조건의 시나리오를 더합니다. 긴급 요청이 들어왔을 때의 Given, When, Then과 기대값이요.

셋째 단계에 가서야 구현해요. 스킬의 절차나 해당 기능의 코드를 명세에 맞춰 고치죠.

마지막 단계는 전체를 다시 실행하는 겁니다. 새 시나리오만이 아니라, 일반 요청, 필수 정보가 빠진 요청을 확정하지 않는 보류, 거절의 기존 시나리오까지 전부요.

순서가 왜 이렇게 중요할까요? 구현 결과만 보고 기대값을 정하면, 잘못된 행동도 정답으로 옮겨 적을 위험이 있습니다. 구현 뒤에 시험을 보완하더라도 원래 요구와 근거에서 기대를 정해야 해요. 기대값을 먼저 적어 둬야, 구현이 그걸 맞혔는지 확인할 수 있어요.

[SLIDE 48] 실행할 때마다 작동하는 장치와 바꿀 때마다 하는 검증

지금까지 본 장치들은 작동하는 때가 둘로 나뉩니다. 실행할 때마다 작동하는 것, 그리고 바꿀 때마다 하는 것이에요.

화면 왼쪽이 실행할 때마다 작동하는 장치입니다. 출력 계약의 검사, 가드레일, 사람의 승인이죠. 에이전트가 추천안을 낼 때마다 그 한 건을 검사하고 걸러요. 정해 둔 조건을 어긴 결과를 막는 역할입니다. 가능한 모든 잘못을 잡는 것은 아니에요.

오른쪽은 바꿀 때마다 하는 검증이에요. 시나리오의 실행, 판단 채점, 전체 회귀. 스킬과 명세, 코드뿐 아니라 모델이나 지식, 중요한 설정을 바꿨을 때도 관련 검증을 돌립니다. 찾는 건 변경 때문에 달라진 행동이고요.

오른쪽 둘째 줄의 판단 채점은 이때 바꾸기 전과 후의 실행을 나란히 놓고 볼 수 있어요. 항목마다 점수가 어떻게 달라졌는지, 그리고 그 사이에 지식 그래프에서 끊기거나 새로 생긴 관계가 무엇인지까지 보여 줍니다. 그래서 점수가 떨어졌을 때, 원인 후보가 될 변경을 좁힐 수 있습니다. 동시에 조회 자료도 달라졌다면 지식 변경 때문이라고 단정할 수 없으므로, 비교 조건을 맞춰 다시 확인해야 해요.

여기서 기억해 둘 게 있어요. 한쪽이 있다고 다른 쪽을 빼면 안 됩니다. 승인 단계가 있으니까 회귀 시험은 필요 없다고 해 볼까요? 그러면 변경 때문에 추천의 질이 떨어져도, 그걸 사람이 매번 떠안게 돼요. 반대로 회귀 시험을 다 통과했으니 승인은 필요 없다고 하면요? 시험이 다루지 않은 상황에서, 잘못된 한 건이 그대로 실행됩니다.

[SLIDE 49] HYD의 세 에이전트가 내는 산출물

앞에서 승인과 거절의 시험을 세 흐름에 적용한다고 했죠. 그 세 흐름을 맡은 에이전트가 화면의 세 줄입니다. 같은 방식으로 만든 에이전트라도, 맡은 일이 다르면 내는 산출물이 달라져요.

먼저 가운데 하늘색 구획을 보세요. 셋에 공통인 틀이에요. 목표가 있고, 스킬과 도구가 있고, 출력 계약이 있죠. 틀은 같습니다.

다른 건 그 틀에 넣은 거예요. 맨 위 줄은 쿨러 성능 저하 경보를 맡은 냉각 긴급 대응입니다. 오른쪽 끝을 보면, 팬 최대 운전과 대안 셋을 담은 추천안을 내죠. 가운데 줄은 정기 정비 도래를 맡은 정기 정비 계획이에요. 언제 정비할지를 정한 정비 시간을 냅니다. 맨 아래 줄은 예비품 재고가 기준을 벗어났을 때의 예비품 구매고요. 공급사와 수량과 금액을 담은 발주안을 냅니다.

넣은 게 다르다는 건 스킬만 얘기하는 게 아니에요. 붙은 도구도 다릅니다. 냉각 긴급 대응은 원인을 진단하고 조치 후보를 평가하는 판단 도구를 써요. 정기 정비 계획은 정비 기한과 허용 한계까지 남은 시간, 예정된 야간이나 주말 정비 시간을 읽는 도구를 쓰고요. 예비품 구매는 재고와 공급사 견적을 읽는 도구를 씁니다. 업무가 다르니 읽을 값과 지킬 규칙이 다르고, 그래서 산출물의 항목도 달라지는 겁니다.

그러니까 새 업무를 에이전트에 맡길 때도 틀부터 새로 만들 필요는 없어요. 공통 구조를 활용하되 목표와 스킬, 도구와 권한, 출력 계약을 그 업무에 맞춰 확인해야 합니다. 필요한 연결이나 실행 기능이 없으면 구현도 더해야 하고요. 산출물이 달라지는 건 틀이 달라서가 아니라, 그 자리에 넣은 게 달라서예요.

[SLIDE 50] 실습 산출물과 근거를 다루는 방식

에이전트를 만들어 본 실습들에서 나온 산출물 다섯을 표로 모았습니다. 이번엔 오른쪽 칸, 근거를 다루는 방식을 중심으로 보세요.

첫 줄 점검 계획은 확인되지 않은 원인을 가설로 표시했습니다. 품질 대응 카드는 원인이 정해지지 않았으면 단정하지 않고 조사 요청으로 냈고요. 도표와 이상 구간 요약은 두 값이 함께 움직였다는 것만으로 원인이라고 단정하지 않았어요. 작업 목록은 담당자와 기한이 없으면 미정으로 적었고, 학습 코치는 정답에 출처를 붙여서 되짚어 줬죠.

형식은 계획, 카드, 도표, 목록, 피드백으로 다 다릅니다. 그런데 오른쪽 칸엔 공통점이 있어요. 모르는 건 확정하지 않고, 아는 것에는 출처를 붙인다는 겁니다. 산출물의 모양은 응용마다 달라져도, 근거를 다루는 이 방식은 달라지지 않아요.

실습에서 만든 인수인계서도 같은 방식이었죠. 한 출처를 읽지 못했으면 빈칸으로 두지 않고, 읽지 못했다고 적었습니다. 이 방식이 왜 그렇게 중요할까요? 산출물을 받는 쪽은 사람이든 시스템이든, 적힌 대로 믿고 다음 일을 시작하거든요. 확정하지 않은 걸 확정처럼 적으면, 그 잘못이 다음 일로 그대로 넘어갑니다. 반대로 확인하지 못했다고 적혀 있으면, 받는 쪽이 거기서 멈추고 확인할 수 있죠.

[SLIDE 51] 산출물의 형식을 정하는 것: 읽는 쪽과 이어지는 일

산출물의 형식은 누가 읽고 그 뒤에 어떤 일을 하는지에 맞춰 정합니다. 인수인계서는 다음 담당자가 읽고, 추천안은 승인 화면을 만드는 시스템과 승인할 사람이 함께 읽습니다.

사람에게는 맥락과 근거, 미확인 부분이 잘 보이는 문서와 도표가 도움이 됩니다. 시스템과 주고받을 때는 항목의 이름과 형식, 값의 뜻을 약속해야 해요. HYD는 그 전달에 JSON을 사용하지만, 모든 시스템이 같은 형식을 쓰는 것은 아닙니다.

값에도 조건이 있습니다. 예를 들어 안을 구별하는 번호가 중복되면 사람이 고른 안과 시스템이 처리할 안을 명확히 연결할 수 없죠. HYD의 출력 계약은 안마다 다른 구분 값을 요구합니다. 사람만 읽는 문서라도 이런 모호함을 그대로 넘겨서는 안 됩니다.

추천안은 승인과 실행 검토로, 품질 대응 카드는 조사로, 인수인계서는 다음 교대 작업으로 이어집니다. 그래서 형식 검사에 더해 내용과 근거도 확인해야 합니다. 읽는 쪽이 필요한 결정을 정확히 이어 갈 수 있게 만드는 것이 출력 계약의 목적입니다.

[SLIDE 52] 검증 실습의 산출물이 쓰이는 곳

실습의 산출물은 그 실습에서 끝나지 않고 다음 일로 이어집니다. 화면 왼쪽 회색 구획이 사람의 결정과 명세를 다룬 실습의 산출물이고, 가운데 하늘색 구획이 시나리오와 회귀를 다룬 실습, 그러니까 이번 회차에 해 본 실습의 산출물이에요.

왼쪽 맨 위의 확인 질문과 전후 기록은 필수 정보가 빠진 경우의 시나리오가 됐습니다. 그 아래 spec.md와 기대 결과는 시나리오의 재료가 됐고요. 명세 실습에서 저장해 둔 입력과 기대값이 바로 시나리오의 Given과 Then이 된 거예요.

가운데 시나리오는 실행돼서 회귀 기록으로 남습니다. 그리고 오른쪽 화살표처럼, 그 기록이 다음에 요구가 바뀔 때 다시 실행할 기준이 되죠.

왼쪽 맨 아래의 초안, 검토 의견, 수정본은 에이전트끼리의 검토와 사람의 승인을 서로 다른 항목에 남기는 연습이었어요. AI의 검토 의견을 자동 검사나 사람의 승인 기록과 혼동하지 않게 구별한 겁니다. 가운데 아래 인수인계서는 근거 시각과 함께 다음 교대의 담당자에게 넘어가고요. 값마다 어디서 언제 읽었는지를 붙이는 건, 근거가 놓인 자리에서 본 현재값의 원칙 그대로입니다.

[SLIDE 53] 운영 중 결과가 기대와 다를 때 먼저 살펴볼 곳

이번엔 에이전트를 운영하다가 결과가 기대와 다를 때를 생각해 볼게요. 증상이 가리키는 장치부터 살펴보면, 고치지 않아도 될 곳을 건드리지 않습니다. 표의 왼쪽 칸이 증상, 가운데가 먼저 살펴볼 곳이에요.

첫 줄, 값이 실제와 다르면 도구와 실행 기록을 봅니다. 호출 기록에서 무엇을 물었고 무엇을 받았는지, 값이 얼마나 묵었는지를 확인해요. 호출 기록부터 보는 데는 까닭이 있습니다. 도구를 부를 때마다 입력과 응답, 시각과 걸린 시간이 한 줄씩 남거든요. 그래서 값이 이상할 때, 도구가 애초에 그 값을 돌려줬는지, 아니면 받은 값과 다르게 옮겨 적었는지가 여기서 갈립니다. 어느 쪽이냐에 따라 고칠 곳이 달라지니까, 짐작으로 고치기 전에 기록부터 보는 거죠.

둘째 줄, 필요한 값을 받지 못했다면 도구를 호출했는지부터 보고, 서버 연결과 권한, 오류 응답을 차례로 살펴봅니다.

셋째 줄, 묻지 않고 확정했다면 금지 조건과 사람의 개입 지점, 그러니까 명세와 HITL 쪽이에요. 마지막 줄, 고친 뒤에 기존 요청의 결과가 달라졌다면 회귀 시험에서 실패한 시나리오를 봅니다.

검증 단원에서 본 「실패한 시험에서 고칠 곳」 표는 시험이 실패한 데서 출발했죠. 이 표는 운영 중에 본 결과에서 출발합니다. 출발점은 달라도 생각하는 방식은 같아요. 증상을 출발점으로 기록을 대조하고, 확인한 원인에 맞춰 고치는 겁니다.

[SLIDE 54] 사람이 정하는 기준과 에이전트가 정하는 과정

마지막으로 사람이 정하는 기준과 에이전트가 실행 중 선택하는 과정을 나누어 보겠습니다. 왼쪽은 목표와 지침, 도구 권한, 계약과 금지 조건, 시나리오의 기대값입니다. 업무를 맡기기 전에 합의하고 관리할 기준이에요.

오른쪽은 그 기준 안에서 어떤 도구를 어떤 입력으로 부를지, 받은 결과에 따라 무엇을 더 확인할지, 어떻게 설명할지입니다. 서로 독립적인 조회의 순서나 설명의 문장은 달라질 수 있습니다. 하지만 필요한 확인을 건너뛰거나 권한을 넘어서는 선택까지 허용되는 것은 아니죠.

왼쪽 기준도 영원히 바뀌지 않는 것은 아닙니다. 요구가 바뀌면 근거를 검토하고 명세와 검사를 함께 바꾸며, 어느 버전에 어떤 기준을 적용했는지 남깁니다. 실행 결과에 맞춰 아무 설명 없이 기준을 바꾸는 일과는 다릅니다.

앞의 쿨러 기록에서 재조회를 하겠다는 말과 실제로 확인된 호출을 구별한 것도 같은 이유입니다. 계획만으로 수행을 인정하지 않고, 기록이 보여 주는 범위에서 판단하는 거예요.

맡길 수 있는 범위는 멋진 답변 한 번으로 정해지지 않습니다. 기대와 권한을 정하고, 실제 결과와 남은 불확실성을 확인하고, 바꾼 뒤 다시 검사하는 과정을 통해 근거를 쌓아 갑니다. 이제 그 흐름을 한 가지 금지 조건에 대어 복습해 보겠습니다.

[SLIDE 55] 금지 조건 한 줄이 지켜지는 자리

그 겹겹의 장치가 실제로 어떻게 맞물리는지, 복습하면서 확인해 보겠습니다. 이번 회차에 배운 흐름을 「거절하면 설비 명령이 없다」는 금지 조건 한 줄에 대어 볼 거예요. 표의 다섯 줄은 이 한 줄이 놓이는 자리와, 그 자리에서 어떤 모양이 되는지를 보여 줍니다. 먼저 위에서부터 같은 한 줄이 줄마다 어떤 모양이 됐는지 대어 보고, 넷째 줄 회귀와 다섯째 줄 실행 중 장치가 각각 언제 작동하는지 가려 보세요.

위의 세 줄부터 볼게요. 명세에서는 금지 조건과 수용 기준의 문장입니다. 구현하기 전에 정하죠. 시나리오로 오면 Then이 됩니다. 설비 명령 0건, 업무 처리 0건, 거절로 닫힌다. 실행 결과로 가릴 수 있는 사실로 바뀌었죠. 시험에서는 단언이 돼요. 명령 번호가 없고 업무 처리 목록이 비어 있는지를, 시험을 돌릴 때마다 확인합니다.

넷째 줄 회귀는 바꿀 때마다 작동합니다. 긴급 조건을 더한 뒤에 거절 시나리오를 다시 돌려서, 변경 때문에 거절의 행동이 달라지지 않았는지 찾죠.

다섯째 줄은 실행 중이에요. 가드레일 검사, 그리고 담당자의 승인과 거절은 추천안이 나올 때마다 작동합니다. 지금 이 한 건이 잘못 실행되는 걸 막는 거예요. 고른 답이 이렇게 갈렸다면 맞게 가린 겁니다. 같은 한 줄을 서로 다른 때에 지키는 두 장치인 거죠.

[SLIDE 56] 오늘 다룬 핵심 개념과 주요 용어 정리

이번 회차의 용어를 연결해 보겠습니다. BDD는 구체적인 행동 사례로 기대를 함께 정하고 실행 가능한 검증에 연결하는 개발 방식입니다. 시나리오는 그 상황과 기대를 적은 예이고, Given, When, Then은 전제와 행동, 기대 결과를 나누는 구조입니다. Gherkin은 이런 시나리오를 적는 언어예요.

단언은 실행 결과가 기대 조건을 만족하는지 확인하는 코드입니다. 단순한 값의 일치뿐 아니라 범위나 항목의 유무를 검사할 수도 있어요. 경계값 분석은 판정이 갈리는 기준 주변을 확인하고, 회귀 시험은 변경 뒤 의도하지 않은 영향이 생겼는지 기존 검증으로 다시 확인합니다.

결정론적이라는 말은 같은 입력과 조건에서 같은 결과가 나온다는 뜻입니다. 데이터 패브릭은 여러 출처의 데이터를 연결하고 관리해 활용하는 접근이고, 이번 실습은 그중 출처와 시각을 유지하며 함께 읽는 부분을 다뤘습니다. 각각의 이름보다 어떤 문제를 해결하려고 쓰는지 이어서 기억해 주세요.

[SLIDE 57] Review Quiz: 핵심 흐름 되짚어보기

뜻을 알았으면, 그 개념으로 직접 판단할 수 있는지도 봐야겠죠. 퀴즈 세 문제로 확인해 보겠습니다. 문제마다 보기를 읽고 나서 잠깐 멈출 테니, 그때 답을 골라 보세요. 정답과 까닭은 세 문제를 다 본 뒤에 같이 확인하겠습니다.

[SLIDE 58] Quiz 1. 새 시나리오만 통과한 변경

첫 번째 문제입니다. 긴급 정비 조건을 더했어요. 긴급 시나리오는 통과했고, 일반 요청과 거절의 시나리오는 아직 다시 돌리지 않았습니다. 지금 내릴 수 있는 판단으로 가장 알맞은 건 무엇일까요?

1번, 새 기능이 통과했으니 변경을 내보낸다.
2번, 긴급 시나리오를 열 번 더 실행해 평균을 본다.
3번, 기존 시나리오를 모두 다시 실행한 뒤에 판단한다.
4번, 일반 요청의 기대값을 긴급 조건에 맞추어 고친다.

[SLIDE 59] Quiz 2. 두 실행의 차이

두 번째 문제입니다. 같은 종류의 경보 두 건을 처리했어요. 둘째 실행은 설명 문장과 도구를 부른 순서가 첫째와 달랐고, 결과에 판단 번호 항목이 없었습니다. 설명의 의미와 근거는 맞고, 도구 순서는 필요한 권한과 의존 관계를 지켰습니다. 이 조건에서 시험이 실패로 세어야 하는 건 무엇일까요?

1번, 판단 번호 항목이 빠진 것.
2번, 설명 문장이 첫 실행과 다른 것.

3번, 도구를 부른 순서가 다른 것.
4번, 세 가지 모두.

[SLIDE 60] Quiz 3. 실패한 무응답 시나리오

마지막 문제입니다. 명세에는 「무응답이면 지연 알림만 보내고 기다린다」고 적혀 있어요. 그런데 기능을 고친 뒤에 돌려 보니, 10분 뒤에 처리 건이 거절로 닫혔습니다. 무응답 시나리오가 실패한 거죠. 맞는 조치는 무엇일까요?

1번, 시나리오의 기대값을 거절로 닫힌다로 고친다.
2번, 무응답 시나리오를 시험 목록에서 뺀다.
3번, 지연 시간을 30분으로 늘려 다시 실행한다.
4번, 기대값은 그대로 두고 고친 기능에서 원인을 찾는다.

[SLIDE 61] 정답과 판단 기준

세 문제 모두 답을 골라 두셨으면, 정답을 확인해 볼게요. 고른 답과 그 근거를 같이 대어 보세요. 첫 번째 문제의 정답은 3번입니다. 긴급 시나리오가 통과했다는 건 긴급 조건이 동작한다는 것만 알려 줘요. 일반 요청과 거절이 그대로인지는 그 시나리오들을 다시 돌려 봐야 압니다. 변경 뒤에 기존 시나리오를 모두 다시 실행하는 것, 바로 회귀 시험이죠.

1번은 새 행동만 보고 기존 행동은 확인하지 않았습니다. 2번은 같은 긴급 시나리오만 되풀이하니까, 몇 번을 돌려도 기존 행동에 대해서는 아무것도 알려 주지 않아요. 4번은 통과시키려고 기존 기대값을 바꾼 거고요.

두 번째 문제는 1번입니다. 판단 번호는 출력 계약이 정한 항목이라, 빠지면 계약 위반이에요. 문제에서는 설명의 의미와 근거가 맞고, 바뀐 호출 순서도 필요한 조건을 지켰다고 했습니다. 그래서 그 차이 자체는 실패가 아닙니다. 순서가 권한이나 선행 조건을 어겼다면 판단은 달라져요.

세 번째 문제는 4번이에요. 기대값은 명세에서 옵니다. 명세는 그대로 「기다린다」인데 기능이 거절로 닫았으니, 고칠 건 기대값이 아니라 고친 기능이죠. 1번과 2번은 시험을 통과시키려고 기준을 바꾸거나 없앤 거고, 3번은 명세에 없는 값으로 조건을 바꾼 겁니다. 그렇게 하면 시험은 통과해도, 명세와 행동은 어긋난 채로 남아요.

여기까지가 이번 회차입니다.

다음 회차에서는 에이전트 하나의 일을 넘어서, 사람과 에이전트와 시스템의 작업들을 하나의 업무 프로세스로 묶어 봅니다. 누가 어떤 순서로 무엇을 맡는지를 공통된 그림으로 그리고, 그 그림을 실제로 실행해 보는 방법을 다룰 거예요. 수고 많으셨습니다. 감사합니다.